

Phase 2 Feasibility Study

Intelligent AI Powered Circular Campus Resource Exchange and Asset Life Cycle Management System

Mohammed Alyammahi 100060927

Mohammed Ali 100060689

Sanad Alshehhi 100059950

Group 1-B2 - Fall 2026 - 29 September 2026

Instructors:

Ala' Alsamarneh
Hamdan Alshkeili

We used AI to assist with the report

Table of Contents

1 Introduction and project scope	3
Expected Benefits of the Project	3
2 Market Analysis	3
Snipe-IT	3
ManageEngine AssetExplorer	4
Asset Panda	4
Implications for the Proposed System	4
3 Stakeholders and operational feasibility	5
4 Demand and alternative solutions	5
5 Technical feasibility	6
6 Task Distribution	7
7 Data and AI feasibility	8
8 Financial feasibility	8
Development Cost	8
Hosting and Development Tools	8
AI Costs	9
Total Costs	9
Savings, ROI, and Payback	9
9 Legal security and policy constraints	9
10 Schedule and resource feasibility	10
How We Will Complete the Weekly Tasks	10
11 Main risks assessment	11
12 Mitigation Plan	12
13 Recommendation and immediate actions	12
References	13

1 Introduction and project scope

This report evaluates whether our Circular Campus system can be developed as a student prototype within the semester. We assess technical, financial, operational and schedule feasibility, together with data, AI and policy constraints. The assessment follows the Phase 2 presentation and the project description.

The proposed system will help departments find and reuse available resources, including computers, furniture, laboratory equipment and office materials. It will manage asset records, requests, reservations, approvals, transfers, maintenance, user roles, notifications and reports. AI will support classification, matching, sustainable actions, a campus assistant and report summaries.

Expected Benefits of the Project

The project is expected to help university departments reuse available assets, reduce unnecessary purchases and limit waste. Keeping asset information in one system will make it easier to find items, check their condition and availability, and track requests, approvals and transfers.

AI functions will help users find suitable assets and consider options such as reuse, repair or recycling. Clear records and reports will support accountability and help the university monitor asset use and potential savings. These benefits will be evaluated through prototype testing and user feedback, while actual financial and environmental savings will require evidence from campus use.

Scope limits: The prototype will demonstrate the main asset-management workflow and all five required AI functions (The five required AI functions are asset classification, matching and ranking, sustainable action advice, the campus assistant, and AI reporting.) using sample campus data. Testing will involve a limited number of assets and users. Full campus deployment, integration with existing university systems and training a custom AI model are outside the prototype's scope. We modified the scope from phase 1 by specifying the AI functions.

Working assumptions: Three team members will use existing laptops, sample asset records and an approved AI service. Each member plans to contribute about eight hours per week over nine weeks, providing approximately 216 team hours. Tool access, sample-data availability and the proposed budget must be confirmed. Actual campus adoption and financial savings will need evaluation after the prototype is tested.

Preliminary decision: Proceed with a controlled prototype, provided that the technical trial, data preparation and workload review support the plan.

2 Market Analysis

The proposed Intelligent AI Powered Circular Campus Resource Exchange and Asset Life Cycle Management System focuses on university resource sharing and reuse. Three relevant solutions—Snipe-IT, ManageEngine AssetExplorer, and Asset Panda—provide benchmarks for asset records, custody tracking, maintenance, and reporting [1], [3], [5].

This analysis compares publicly available vendor information with our proposed requirements. It does not represent hands-on testing or confirmed findings about campus practices.

Snipe-IT

Snipe-IT is an open-source system for tracking equipment, locations, and responsible users. Its features include check-in and check-out, custom fields, barcode and QR code support, warranty and licence

notifications, asset history, and requests for available items. These functions overlap with our inventory, custody, and resource request requirements [1].

Its main strengths are flexibility and accessible pricing. Self-hosted software is free, while paid hosting and support are available [2]. However, self-hosting requires technical resources for installation, security, backups, and administration.

Snipe-IT provides a useful model for accurate records and asset responsibility. Our proposed additional focus is matching surplus resources against minimum and recommended requirements, explaining rankings, combining quantities across departments, and reporting shortages. The reviewed documentation does not establish this complete matching workflow as a standard capability, so customization would require further evaluation [1].

ManageEngine AssetExplorer

ManageEngine AssetExplorer manages hardware and software throughout their life cycle. It supports automated discovery, inventory management, software licence tracking, purchase and contract management, reporting, and configurable workflows. It also records relationships between IT components and services and offers cloud and on-premises deployment [3].

Its strength is the depth of its IT management functions. Automated discovery helps maintain device information, while software usage and licence records support purchasing decisions. This makes it relevant to university IT departments [3]. Commercial pricing depends on the offering and number of managed IT assets [4].

Our proposed system emphasizes campus exchange across computers, furniture, laboratory equipment, books, and office materials. It connects departmental demand with verified surplus resources and approved reuse. Whether AssetExplorer could support this complete process through configuration or integration requires further assessment.

Asset Panda

Asset Panda is a configurable platform with a dedicated higher education offering. It tracks equipment and furniture across departments and supports loans, reservations, approvals, transfers, mobile barcode scanning, inspections, and lifecycle records. It also helps identify idle equipment before additional purchases, making it a close competitor [5].

Its strengths include varied asset categories and adaptable fields, workflows, and reports. Asset Panda also advertises Ursa AI features for inspections, asset information, and reporting [6]. Therefore, AI alone would not distinguish our project.

Our opportunity is to develop a specific matching process that checks minimum requirements, ranks assets against preferences, explains recommendations, and keeps partially fulfilled requests open. These requirements should be checked through a product demonstration because Asset Panda already overlaps with several proposed functions. Its suitability for our budget would require a quotation [7].

Implications for the Proposed System

The comparison shows that inventory, transfers, and lifecycle tracking are established capabilities [1], [3], [5]. Our project combines ideas from Snipe-IT's inventory and custody tracking and ManageEngine's lifecycle management around the university's resource exchange needs.

The project will prioritize explainable matching, shortage reporting, availability notifications, and repair-before-replacement decisions. For example, if five laptops are requested but only three qualify, the system should explain their suitability and retain the remaining demand for two.

Financial and sustainability reports will connect estimated savings, waste diversion, and carbon reductions to completed exchanges and supporting evidence. These are proposed benefits, not proven advantages. Stakeholder interviews and prototype evaluation will help determine whether a dedicated system is needed or an existing product could meet campus requirements.

3 Stakeholders and operational feasibility

The workflow is practical if each department keeps its records current and authorized officers take responsibility for approvals. Stakeholder discussions should check whether the following responsibilities fit actual campus practice.

Stakeholder	Operational need
Requesters	Find suitable items, state minimum and preferred requirements, and track requests.
Department representatives	Confirm departmental needs and review the release or receipt of resources.
Asset Management Team	Maintain listings and records. Authorized members approve transfers and end-of-life actions.
Maintenance staff	Record inspections and repairs, and confirm whether an item is safe to use.
Finance and sustainability officers	Review purchase evidence, costs and supported sustainability estimates.
System administrator	Manage accounts, permissions, reference data and backups.

Adoption approach: Our approach is to communicate with Office of Facilities Management at Khalifa University and ask them questions regarding the project to get their feedback.

Operational finding: We scheduled a meeting with Office of Facilities Management and it is currently in progress.

4 Demand and alternative solutions

Our target users are university departments and support teams. The expected need is easier internal reuse and better asset tracking. Actual demand is not yet measured. Interviews and a review of sample surplus items and unmet requests will test this need.

Alternative	Assessment for this project
Shared spreadsheet and email	Quick to set up, but approval tracking, record updates and matching would remain largely manual.
Existing asset platform	For instance, Snipe-IT provides asset history, requests, permissions and an API [1]. We would need to assess and adapt it for the required AI and campus workflows.
Custom campus prototype	Fits the course functions and allows a direct comparison of conventional and AI methods, but needs more development and testing.

The comparison supports a custom prototype for the course. Existing-platform capabilities are documented in Snipe-IT product features [1]. The customization assessment is our preliminary judgment.

5 Technical feasibility

We plan to use HTML, CSS, and JavaScript to build the website, Python with Django to run its functions, and PostgreSQL to store the data. We will use Visual Studio Code to write code and GitHub to share our work. ChatGPT (Codex) will help us with coding, fixing errors, and creating tests. Each member will check and test the code before using it.

The system will let users add assets, search for items, send requests, approve transfers, and view maintenance records and reports. AI will help classify assets, suggest suitable items, and answer questions. Users will have different permissions based on their roles and departments. Basic features will still work if the AI service is unavailable.

We will use clearly labelled sample data for assets, users, departments, and requests to test the prototype on our laptops and university computers. The main challenges are time, AI costs, and coding errors. We will manage these through weekly progress checks, a spending limit, backups, and regular testing.

Resources and skills: Use existing laptops and lab facilities, GitHub for shared work, and a local or approved hosted deployment. The team needs web programming, SQL, API integration and testing skills. Trial login, asset registration and one AI request before committing to the stack. Use sample file imports and exports for external integrations.

6 Task Distribution

Previously, we did not specify the task distribution because we were missing one member. The task distribution as follows:

Week	Mohammed Ali: Requirements and Interface	Mohammed Salem: Database and Core Functions	Sanad Alshehhi: AI and Reporting
Week of 5 October — Phase 3 submission	Gather stakeholder requirements, document workflows, and define acceptance criteria.	Define asset data, permissions, reservation rules, and transfer requirements.	Define AI requirements and prepare sample assets and requests.
Week of 12 October	Design wireframes for listings, search, requests, and dashboards.	Design database tables, relationships, and APIs.	Design classification and matching logic using minimum and recommended requirements.
Week of 19 October	Develop navigation and reusable interface components.	Design authentication, department access, audit records, and reservation controls.	Design the campus assistant, sustainability advice, and report generation.
Week of 26 October — Phases 4 and 5 submission	Finalize interface designs and role-specific workflows.	Finalize architecture, database diagrams, and backup procedures.	Finalize AI integration, human review, and fallback designs.
Week of 2 November	Build asset registration, listings, search filters, and details pages.	Implement accounts, permissions, asset records, and maintenance history.	Implement classification and minimum-requirement eligibility checks.
Week of 9 November	Build request, reservation, approval, and transfer screens.	Implement reservations, approvals, transfers, receipt confirmation, and notifications.	Implement matching explanations, shortage tracking, and the campus assistant.
Week of 16 November — Phase 6 submission	Connect screens to backend functions and develop dashboards.	Integrate workflows, audit history, and backup recovery.	Integrate sustainability advice and report summaries; test AI fallback behavior.
Week of 23 November — Phases 7 and 8 submission	Conduct usability testing and prepare the user guide and slides.	Test permissions, reservation conflicts, transfers, and recovery.	Evaluate AI accuracy, verify report evidence, and prepare the AI demonstration.

7 Data and AI feasibility

Use an existing pretrained AI service rather than train a large model. For matching, compare the meaning of descriptions using text embeddings, after checking minimum specifications, quantity and availability. The trial must confirm support for the required text, image and document inputs.

AI function	Feasible prototype approach
Asset classification	Suggest categories and tags from asset details and supported attachments. Users accept or correct them.
Matching and ranking	Filter ineligible assets, rank suitable matches and explain the main reasons. Compare with keyword search.
Sustainable action advice	Recommend reuse, repair or other eligible actions using condition and cost evidence. Officers make the decision.
Campus assistant	Answer from authorized records and approved guidance. Require confirmation before submitting a request.
AI reporting	Summarize verified totals and activity. Calculate financial and environmental figures with conventional code.

Data needs: Asset identity, specifications, quantity, ownership, condition and location, plus request, maintenance, transfer and purchase records. Use labelled sample records initially, standardize units and categories, remove duplicates and flag missing details. Restrict personal and financial information.

Feasibility finding: Achievable at prototype scale if service access and representative test data are available. Keep separate test examples, review incorrect suggestions and show uncertainty. Retain manual search and normal workflows during an AI outage. Report savings and carbon estimates only when their assumptions and supporting evidence are available.

8 Financial feasibility

The project's costs include development, AI usage, hosting, and maintenance. Provider prices are referenced below, while working hours, usage levels, and expected savings are planning estimates. USD prices are converted using approximately AED 3.67 per dollar, excluding taxes.

Development Cost

Our work plan estimates 216 team-hours for requirements, design, coding, testing, and documentation. We use AED 100 per hour, informed by some suppliers on Clutch's UAE pages advertising USD 25–49 per hour [8]. This gives $216 \times \text{AED } 100 = \text{AED } 21,600$. It represents the value of student work, not wages we expect to pay. ChatGPT (Codex) will assist with coding, but team members will review and test the code.

Hosting and Development Tools

We propose a DigitalOcean Basic Droplet costing USD 12 per month, with 2 GiB memory, one virtual CPU, and 50 GiB storage. Weekly backups add 20%, bringing the combined yearly cost to approximately AED 634 [9]. We choose this option because Django and PostgreSQL can run on one server. Our annual infrastructure allowance is AED 1,200, leaving room for upgrades. The team will manage server security, updates, and recovery.

We also considered Render, which lists an entry-level paid web service at USD 7 per month, with database resources priced separately [10]. DigitalOcean provides a straightforward single-server budget for the prototype. We will use PostgreSQL, which has no software licence fee [11], and GitHub Free for sharing code and tracking changes within its included limits [12].

AI Costs

OpenAI lists GPT-6 Astra's standard short-context API prices at USD 10 per million input tokens and USD 50 per million output tokens [13]. Assuming five million input tokens and one million output tokens during development, the cost would be USD 100, or approximately AED 367. Adding a 20% extra-usage allowance gives approximately AED 440. These token quantities are assumptions, and additional tool charges are excluded.

These are API prices, not ChatGPT subscription prices. If we use a subscription for coding, we will replace the estimate with its actual cost rather than count the same activity twice. Separately, we budget AED 1,440 per year for the application's AI features: AED 100 per month plus 20%. Prototype testing will help us refine this allowance.

Total Costs

The initial cost is AED 21,600 for labour + AED 440 for development AI + AED 300 for temporary hosting and setup = AED 22,340. Excluding unpaid student labour, initial cash spending is approximately AED 740. Existing laptops and university computers are assumed to be available.

Annual running costs are AED 1,200 for infrastructure + AED 100 for a domain + AED 1,440 for application AI + AED 2,400 for maintenance = AED 5,140. Maintenance assumes 24 hours annually at AED 100 per hour. The setup, domain, and maintenance amounts are planning allowances, not supplier quotations.

Savings, ROI, and Payback

For illustration, we assume annual asset purchases of AED 100,000 and purchasing savings of 15% through reuse. These are not confirmed university figures. Annual savings would be AED 15,000, giving a net annual saving of $\text{AED } 15,000 - \text{AED } 5,140 = \text{AED } 9,860$.

The estimated payback period is $\text{AED } 22,340 \div \text{AED } 9,860 =$ approximately 2.3 years. Over three years, total savings would be AED 45,000 and total costs would be $\text{AED } 22,340 + (\text{AED } 5,140 \times 3) = \text{AED } 37,760$. Therefore, the three-year ROI is $(\text{AED } 45,000 - \text{AED } 37,760) \div \text{AED } 37,760 \times 100 =$ approximately 19.2%.

If savings reach only 10%, payback increases to approximately 4.6 years. These estimates exclude inflation and physical repair or transport costs. We will validate savings through testing and compare supplier quotations before deciding whether our system offers better value than an existing product.

9 Legal security and policy constraints

Data and privacy: Before using real records, confirm the applicable UAE data-protection requirements and university rules for access, retention and external AI processing. Use approved or fictional data during development and minimize information sent to an AI service.

Security and ownership: Apply role and department permissions, secure credentials and keep an audit history. Verify ownership and procurement authority before a transfer. Human officers must approve release, donation, recycling and disposal.

Safety and ethics: Block unsafe items from normal transfer until inspection. Review AI recommendations for unsuitable or unfair matches. Treat uploaded text as data, and prevent it from changing permissions or bypassing approval steps.

Intellectual property: Check software, model and dataset licenses. Use only documents and images the team is permitted to process. Campus use remains subject to the relevant university approvals.

10 Schedule and resource feasibility

Use the Phase 2 presentation as the working schedule. The project description lists later dates, including 4 October for Phase 2, so the instructor must confirm the final deadlines.

Deliverable	Lab week in 2026
Phase 2 feasibility report	29 September
Phase 3 detailed requirements	5 October
Phases 4 and 5 system design	26 October
Phase 6 draft implementation	16 November
Phases 7 and 8 tests and final demonstration	23 November

Planning assumption: We will each contribute about 8 hours per week over roughly nine weeks, providing approximately **216 team hours** ($3 \times 8 \times 9$). The proposed allocation is 24 hours for requirements and feasibility, 30 for design, 76 for core development, 38 for AI integration, 28 for testing and 20 for documentation. We will confirm team availability before committing to this plan.

Split initial work between core data/workflows and interfaces/AI integration, between the three of us and review requirements and tests. Test throughout development, integrate weekly and reserve the final week mainly for fixes and rehearsal. Keep meaningful individual GitHub commits. The schedule is tight and depends on basic versions of all required features.

How We Will Complete the Weekly Tasks

Week of 5 October: We will review Phase One and gather feedback from campus stakeholders to confirm how assets are listed, requested, approved, and transferred. Each member will document requirements for their assigned area. We will combine these into one requirements document, resolve inconsistencies, and define checks for deciding whether each function works correctly.

Week of 12 October: We will translate the requirements into interface wireframes, a database design, and AI processing steps. Using a sample laptop request, we will check that the interface captures the information needed by both the database and matching process. We will agree on common field names and API formats so that members can develop their components in parallel.

Week of 19 October: We will develop reusable interface components and design the controls behind each workflow. We will map which roles can view, edit, request, and approve assets, and define how reservations prevent competing allocations. For AI features, we will specify authorized information sources, expected responses, and how uncertain or unsupported results should be handled.

Week of 26 October: We will combine the interface, database, architecture, and AI designs into the design submission. The team will walk through a complete example from asset registration to confirmed receipt, checking that every step has the required data and permissions. We will also

document error handling, backup recovery, human approval points, and conventional functions that remain available when AI is unavailable.

Week of 2 November: We will implement the first working components using sample campus asset records. Mohammed Ali will build the asset screens, Mohammed Salem will connect them to protected database operations, and Sanad Alshehhi will implement classification and eligibility checks. We will test valid, missing, and incorrect inputs and review generated code before integrating it.

Week of 9 November: We will connect requests to reservations, approvals, and transfers. The team will test cases such as two departments requesting the same asset, a rejected request releasing its reservation, and a transfer awaiting receipt confirmation. AI matching will explain why assets qualify, identify unmet preferences, and report shortages. The campus assistant will be tested to ensure it only accesses information permitted for the user.

Week of 16 November: We will integrate the main functions into a working prototype and run complete workflows. Reports will use recorded asset activity, while savings and environmental estimates will include their supporting data and assumptions. We will test notifications, audit records, backup restoration, and AI service failures, then fix issues that prevent the main features from working before the draft submission.

Week of 23 November: We will complete final testing and collect evidence against the agreed acceptance criteria. Users or labelled proxies will attempt the main tasks, while the team checks permissions, reservation conflicts, classification accuracy, and matching usefulness. We will document results and remaining limitations, finalize the user guide and presentation, and rehearse a demonstration covering resource listing, matching, approval, transfer, and reporting.

All dates are in 2026, with submissions due during the scheduled laboratory sessions. All members will share code review, integration, documentation, and presentation preparation. GPT-6 Astra will serve as the main code-generation assistant, while each member remains responsible for understanding, reviewing, and testing their code.

11 Main risks assessment

The risk scores in this report are preliminary estimates (we will change the numerical measurements throughout the semester).

Likelihood ratings are based on expected challenges, including limited development time, incomplete data, AI service availability and delayed feedback. Impact ratings consider the estimated time needed to resolve a problem and its effects on required features, privacy and approvals.

The cited methodology supports the scoring approach. The individual scores, time limits and testing targets are proposed for our project and still need validation. We will review these assumptions and update the ratings using actual progress, test results and feedback.

Method: We adapt the probability–impact approach described by Burek (2007). [14] This is a semi-quantitative ranking: the scores and targets are provisional project estimates.

$$\text{Risk score } R = \text{likelihood } (L) \times \text{impact } (I) \quad \text{Maximum score} = 9$$

Likelihood: 1 = unlikely, 2 = possible, 3 = likely. Impact: 1 = up to 1 working day of rework; 2 = over 1 and up to 3 days; 3 = over 3 days or failure of a required feature, privacy or approval control. Use the highest applicable impact. These scales are our project definitions.

We will identify risks from requirements and resource limits, agree scores and record reasons, assign response owners, then review progress and test results weekly to update ratings. Higher scores get priority; any unauthorized action must be fixed before the demo.

Example: Incomplete asset information

Some assets may have missing details, such as their condition or specifications, making them difficult to match with requests.

We could estimate the **likelihood as 3 (likely)** and the **impact as 2 (medium)**, assuming that completing the records and retesting would take two to three working days.

$$\text{Risk score} = 3 \times 2 = 6 \text{ out of } 9$$

Risk	L × I
Scope and learning time	$3 \times 3 = 9$
Incomplete or missing data	$3 \times 2 = 6$
AI access or poor suggestions	$3 \times 3 = 9$
Delayed stakeholder input	$2 \times 2 = 4$
Privacy or approval errors	$2 \times 3 = 6$

12 Mitigation Plan

To reduce project delays, we will prioritize required features, divide tasks among the three team members and review progress weekly. Asset records will be checked for missing information, inaccuracies and duplicates before testing. We will test AI services early and check their suggestions against asset requirements. If an AI service becomes unavailable, manual search will remain available while we resolve the problem. Stakeholder feedback will be requested early, with follow-ups for unanswered questions. To reduce privacy and approval risks, we will use fictional data during development, restrict access by user role and test approval permissions. Any identified errors will be corrected and retested before the final demonstration.

13 Recommendation and immediate actions

Conditional go: Proceed with the academic prototype after checking tool access, sample data and team capacity. This week, confirm the budget and responsibilities and run the small technical trial. Before Phase 3, review the approval workflow and update the requirements. Reassess the plan if required AI functions cannot fit the available time or budget. Manual fallback supports continuity but does not replace the required AI deliverables.

References

- [1] Snipe-IT. (n.d.). *Product features*. <https://snipeitapp.com/product>.
- [2] Snipe-IT. (n.d.). *Pricing*. <https://snipeitapp.com/pricing>.
- [3] ManageEngine. (n.d.). *IT asset management software—AssetExplorer*. <https://www.manageengine.com/products/asset-explorer/>.
- [4] ManageEngine. (n.d.). *AssetExplorer pricing*. <https://www.manageengine.com/products/asset-explorer/pricing.html>.
- [5] Asset Panda. (n.d.). *Higher education asset management software*. <https://www.assetpanda.com/products/asset-panda-for-higher-education/>.
- [6] Asset Panda. (n.d.). *Asset management software*. <https://www.assetpanda.com/>.
- [7] Asset Panda. (n.d.). *Get a quote*. <https://www.assetpanda.com/get-a-quote-pro/>.
- [8] Clutch. *Top software developers in the United Arab Emirates*. clutch.co/ae/developers.
- [9] DigitalOcean. *Droplet pricing*. digitalocean.com/pricing/droplets.
- [10] Render. *Pricing*. render.com/pricing.
- [11] PostgreSQL. *Licence*. postgresql.org/about/licence.
- [12] GitHub. *Pricing*. github.com/pricing.
- [13] OpenAI. *API pricing*. developers.openai.com/api/docs/pricing.
- [14] P. Burek, “Collaborative tools and techniques to build the project risk plan,” presented at PMI Global Congress 2007—North America, Atlanta, GA, USA, 2007. Accessed: Sep. 29, 2026. [Online]. Available: <https://www.pmi.org/learning/library/tools-techniques-build-project-risk-plan-7223>

